

Accelerating General Matrix Multiplication Using a Simulated Systolic Array-based ASIC: A Cross-Platform Performance Analysis

Albert Luo (albertlu), Raymond Shen (rsshen)

April 30, 2026

https://github.com/rashen305/15618_project

Contents

1	SUMMARY	2
2	BACKGROUND	3
2.1	Problem Definition	3
2.2	Key Data Structures and Operations	3
2.3	Work and Parallelism Benefits	3
2.4	Parallelism, Dependencies, and Locality	3
2.5	Systolic Arrays for GEMM	4
3	APPROACH	5
3.1	Technologies and Target Platforms	5
3.2	CPU Mapping	5
3.3	GPU Mapping	5
3.4	Systolic-Array Accelerator Microarchitecture	6
3.4.1	Top-Level Hardware Organization	6
3.4.2	Tiled GEMM Decomposition	7
3.4.3	Processing Element Datapath	8
3.4.4	Systolic Array and Wavefront Feeder	9
3.4.5	Controller FSM	10
3.4.6	Ping-Pong Buffers and Double Buffering	10
3.4.7	Dataflow Modes: OS, WS, and IS	11
3.4.8	Memory Hierarchy Model	11
3.5	Cycle-Approximate Performance Estimator	12
4	RESULTS	13
4.1	Experimental Setup	13
4.2	Metrics	13
4.3	CPU and GPU Baseline Results	14
4.4	Accelerator Throughput versus Square Matrix Size	16
4.5	Throughput Across Realistic Workload Shapes	17
4.6	Dataflow Strategy: Output- vs. Weight- vs. Input-Stationary	18
4.7	Double Buffering and Prefetch Overlap	19
4.8	Memory Hierarchy Sensitivity	20
4.9	Roofline Analysis	21
4.10	PE Utilization and Pipeline Fill/Drain Effects	22
4.11	Cycle Decomposition	23
4.12	Scaling with PE Count	24
4.13	Compute-Only Accelerator Throughput	25
4.14	What Limited Speedup	27
4.15	Summary of Results	28
5	REFERENCES	29
6	WORK DISTRIBUTION AND CREDIT	29

1 SUMMARY

We design and evaluate a parameterized, tiled systolic-array GEMM accelerator using SystemVerilog RTL and a cycle-approximate Python performance model. The original project goal was an FPGA systolic-array implementation, but the final evaluation is better framed as an **ASIC-style accelerator design-space study**: the RTL specifies the hardware microarchitecture, while the Python simulator enables large sweeps over array dimensions, K-tile depth, dataflow, buffering, memory latency, and matrix shape without the prohibitive runtime of full VCS simulation for every point. We compare this specialized spatial accelerator model against optimized CPU and CUDA GPU baselines to study how parallelism, locality, memory bandwidth, and pipeline utilization affect GEMM throughput.

2 BACKGROUND

2.1 Problem Definition

We consider dense matrix multiplication (GEMM), computing $C = AB$ where $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, and $C \in \mathbb{R}^{M \times N}$. The inputs are the matrices A and B (and dimensions M, N, K). The output is C .

2.2 Key Data Structures and Operations

- **Data structures.** Two input matrices (A, B) and one output matrix (C). For CPU and GPU we use row-major storage. For the ASIC design we stream tiles into on-chip storage and use a systolic array of processing elements.
- **Key operations.** Multiply-accumulate (MAC) operations that form dot products for each output element

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}$$

2.3 Work and Parallelism Benefits

GEMM performs $2MNK$ floating point operations. For large matrices, the operation is compute intensive but performance depends heavily on memory hierarchy behavior and data reuse. The computation is primarily data-parallel across output elements, with structured reuse across the shared K dimension.

2.4 Parallelism, Dependencies, and Locality

- **Parallelism.** Output elements are independent given A and B , enabling data-parallel computation across i and j .
- **Dependencies.** The dot product for each output element depends on a reduction over k , which can be unrolled or partially accumulated.
- **Locality.** Tiling improves cache and shared-memory reuse. Systolic arrays exploit spatial and temporal reuse by streaming A and B through the array while accumulating locally.

2.5 Systolic Arrays for GEMM

A systolic array is a regular grid of simple processing elements (PEs) that perform local computation while exchanging data only with their nearest neighbors each cycle. For GEMM, a typical mapping uses a 2D $R \times C$ array of PEs where each PE performs a multiply-accumulate (MAC) on a streaming pair of operands and accumulates a partial sum for one output element (or one output tile element).

Wavefront data movement. Instead of repeatedly reading operands from a shared memory hierarchy, A and B values are injected at the array boundaries and shifted across the array in a coordinated, time-skewed “wavefront.” As A_{ik} flows horizontally and B_{kj} flows vertically, the correct pair meets at PE (i, j) in the same cycle, enabling one MAC/PE/cycle in steady state. This creates spatial reuse: one A value can be reused by many columns as it moves right, and one B value can be reused by many rows as it moves down.

Dataflow choices (what stays put). Different systolic dataflows correspond to which values are kept stationary in the PEs versus streamed through:

- **Output-stationary (OS):** keep partial sums for C in PE-local accumulators; stream A and B .
- **Weight-stationary (WS):** keep one operand (often B tiles) in the PEs to reduce repeated movement; stream the other operand.
- **Input-stationary (IS):** keep A tiles in the PEs to reduce repeated movement

These choices trade off boundary bandwidth, on-chip storage, and control complexity while targeting the same arithmetic.

Utilization and pipeline overheads. Systolic arrays behave like a deep pipeline: they have a fill phase (data propagates to interior PEs), a steady-state phase (ideally one MAC per PE per cycle), and a drain phase (final partial sums emerge). Performance therefore depends not only on peak MAC rate but also on utilization (how well $R \times C$ matches tile shapes) and on whether feeding/draining can be overlapped across tiles (e.g., with buffering).

3 APPROACH

3.1 Technologies and Target Platforms

- **Systolic-array accelerator.** SystemVerilog RTL used to define and validate the intended microarchitecture, plus a Python cycle-approximate simulator used to sweep large ASIC-style accelerator configurations. The simulator is not intended to claim a fabricated ASIC result; rather, it is a microarchitectural model for understanding how throughput scales with array size, memory latency, buffering, and dataflow.
- **CPU.** C++ with OpenMP for threading, cache blocking, and compiler vectorization where applicable.
- **GPU.** CUDA C++ with a naive global-memory kernel and a shared-memory tiled kernel.

3.2 CPU Mapping

Kernel Structure. We implement GEMM with loop reordering and tiling. OpenMP parallelizes work across output tiles. Cache blocking improves reuse of A and B submatrices in L1/L2.

Optimization Journey and What Did Not Work Well. Our initial CPU baseline was the reference implementation (**ref**), which uses a straightforward dot-product style loop nest and prioritizes correctness over speed. This version is useful for validation but significantly underperforms optimized variants. We then moved to **cpu_naive** with improved memory access order, followed by **cpu_omp** with tiling and OpenMP parallelization.

The main optimization iteration for CPU was tile-size tuning in **cpu_omp**. At $N = 1024$ (compute-only mode), tile 32 reached about 72.7 GFLOP/s, tile 64 reached about 73.6 GFLOP/s, and tile 128 dropped to about 64.5 GFLOP/s. This showed that larger tiles did not automatically improve throughput; cache behavior and thread scheduling effects made tile 64 a better default than tile 128 for this machine.

3.3 GPU Mapping

Kernel Structure. We implement (i) a naive kernel where each thread computes one element of C , and (ii) a tiled kernel that stages tiles of A and B into shared memory to reduce global-memory traffic.

Optimization Journey and What Did Not Work Well. Our first GPU implementation was a naive global-memory kernel (**gpu_naive**). It performed well relative to CPU but did not fully exploit data reuse. We then implemented a shared-memory tiled kernel (**gpu_tiled**) and swept tile sizes.

At $N = 1024$ in compute-only mode, **gpu_naive** reached about 684.5 GFLOP/s, while **gpu_tiled** reached about 1081.6 GFLOP/s (tile 16) and 1137.3 GFLOP/s (tile 32), i.e., about a $1.58\times$ speedup from naive to tiled for tile 16 and about $1.66\times$ for tile 32. This confirmed that explicit on-chip reuse via shared-memory tiling was a high-impact optimization step.

3.4 Systolic-Array Accelerator Microarchitecture

3.4.1 Top-Level Hardware Organization

The accelerator is organized as a tiled GEMM engine around one or more parameterized systolic-array configurations. The SystemVerilog top level, `systolic_gemm_engine`, instantiates three selectable accelerator slots. Each slot is a complete `tiled_gemm_accelerator` with its own systolic-array dimensions and K -tile depth.

Only one slot is selected for a benchmark run using `i_array_select`; the selected slot drives the shared memory request/response interface and exposes counters such as cycles, compute cycles, memory stall cycles, prefetch cycles, read requests, write requests, loaded tiles, and output tiles. This gives the RTL a convenient way to package multiple array shapes under one top-level interface. For the final performance study, the Python estimator generalizes this idea and sweeps arbitrary (R, C, K_T) tuples directly.

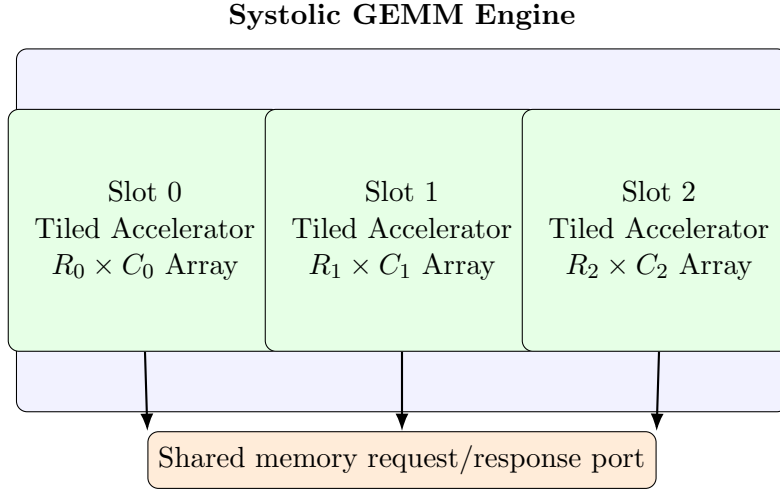


Figure 1: Top-level accelerator organization. The RTL packages three selectable tiled accelerators under one memory interface. The simulator generalizes this to arbitrary array configurations.

Pipeline view. At a high level, the accelerator decomposes into a pipeline with a fill phase, a steady-state phase, and a drain phase. A useful conceptual breakdown is:

1. **Global Memory Access:** Staging tiles from off-chip DRAM to on-chip SRAM buffers.
2. **PE Skewing:** A hardware sequencer that delays inputs to ensure temporal alignment across the spatial array.
3. **Compute Core:** The steady-state MAC pipeline where data reuse occurs at the register level.
4. **Writeback:** Draining the stationary accumulators to the SRAM scratchpad.

3.4.2 Tiled GEMM Decomposition

The accelerator does not attempt to compute the entire matrix at once. Instead, it spatially tiles the $M \times N$ output space by the systolic-array shape and temporally tiles the reduction dimension by K_T :

$$R = \text{NUM_ROWS}, \quad C = \text{NUM_COLS}, \quad K_T = \text{K_TILE}.$$

One output tile has shape $R \times C$. For each output tile starting at global coordinates (m_0, n_0) , the controller iterates over the K dimension in chunks:

$$k_0 = 0, K_T, 2K_T, \dots$$

Each K tile loads

$$A_{\text{tile}} \in \mathbb{R}^{R \times K_T}, \quad B_{\text{tile}} \in \mathbb{R}^{K_T \times C},$$

and contributes

$$C_{\text{tile}} += A_{\text{tile}} B_{\text{tile}}.$$

The PE accumulators are cleared at the beginning of a new C tile, but they are preserved across all K tiles for that same output tile. Only after all K chunks have been consumed does the accelerator write the $R \times C$ output tile back to memory. Therefore, the number of output tiles is

$$T_C = \left\lceil \frac{M}{R} \right\rceil \left\lceil \frac{N}{C} \right\rceil,$$

and the number of K iterations per output tile is

$$T_K = \left\lceil \frac{K}{K_T} \right\rceil.$$

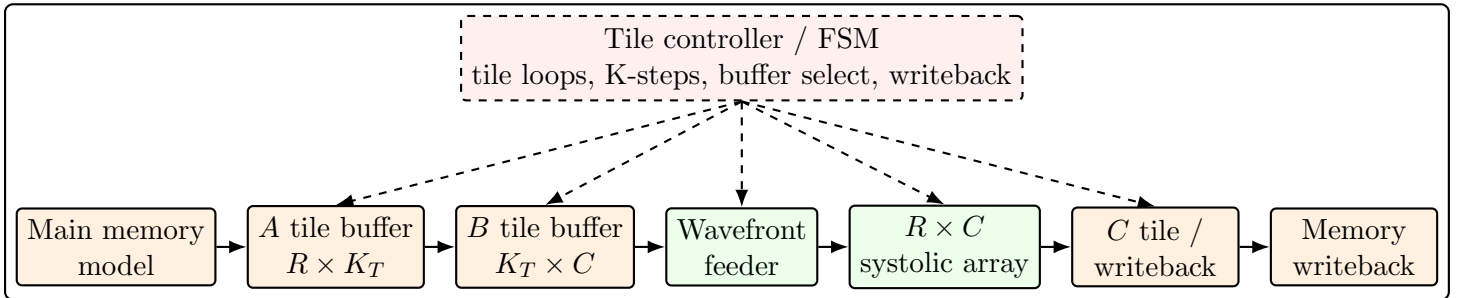


Figure 2: Single tiled GEMM accelerator. Tiles are loaded from memory into on-chip buffers, aligned by the wavefront feeder, processed by the systolic array, and written back after completion of all reduction-dimension tiles.

3.4.3 Processing Element Datapath

The PE is the fundamental compute unit. In the output-stationary case, each PE owns one partial sum for the current C tile. Its datapath contains:

- row input register and row-valid propagation for A values moving left to right,
- column input register and column-valid propagation for B values moving top to bottom,
- a multiplier for the current $A \times B$ product,
- an adder that adds the product into the local accumulator,
- an accumulator register that holds the current partial sum for one output element,
- optional stationary operand registers used by the weight-stationary and input-stationary variants.

A PE performs useful work when the required row and column operands are valid. In output-stationary mode, this requires both the row and column stream to be valid. In weight-stationary mode, a PE can reuse a previously loaded B value. In input-stationary mode, a PE can reuse a previously loaded A value. The PE then forwards operands to its neighboring PEs, producing a nearest-neighbor communication pattern with no global crossbar.

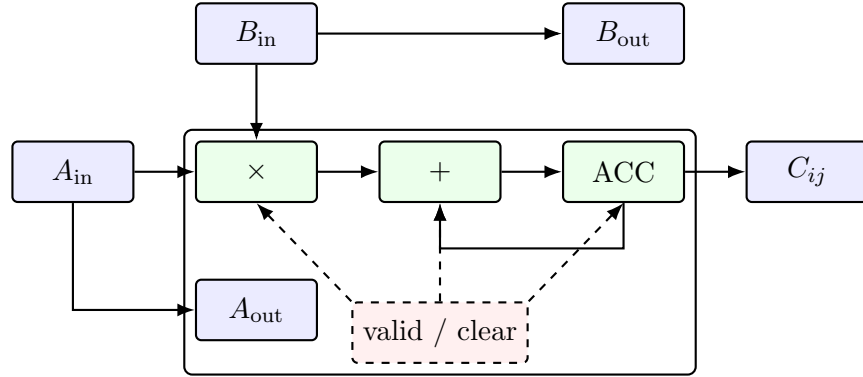


Figure 3: Processing element microarchitecture. Each PE receives streamed operands, performs one multiply-accumulate per active cycle, stores the partial sum locally, and forwards operands to neighboring PEs.

3.4.4 Systolic Array and Wavefront Feeder

The systolic array instantiates an $R \times C$ mesh of PEs. The left boundary receives A values; the top boundary receives B values. The wavefront feeder is responsible for skewing the boundary injections so that the correct operands meet at the correct PE at the correct cycle. Specifically, row value $A[i, k]$ is injected at time

$$t = i + k,$$

and column value $B[k, j]$ is injected at time

$$t = j + k.$$

At PE (i, j) , the A value travels j columns and the B value travels i rows, so both arrive at time

$$t = i + j + k.$$

The feeder therefore converts the dot-product reduction into a spatial wavefront. For a K_T -deep tile on an $R \times C$ array, the active feeder length is

$$K_T + R + C - 2.$$

The $R + C - 2$ term is the fill/drain overhead of the spatial pipeline. For large K_T , this overhead is amortized; for small K_T or small matrices, it can dominate utilization.

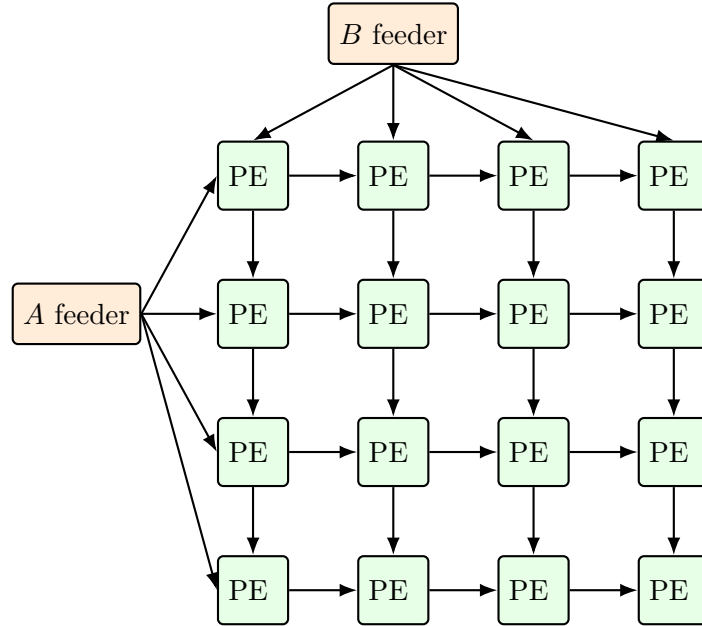


Figure 4: Wavefront-fed systolic array. Elements of the A tile are injected from the left and propagate horizontally across rows, while elements of the B tile are injected from the top and propagate vertically down columns. Each PE performs one multiply-accumulate per active cycle and accumulates one output element of the current C tile.

3.4.5 Controller FSM

The `tiled_gemm_accelerator` wraps the feeder and array with a memory-backed tile controller. Its major states are:

State	Purpose
S_IDLE	Wait for a new GEMM request and latch dimensions/strides.
S_TILE_BEGIN	Initialize a new output tile and clear or preserve buffers depending on dataflow.
S_WAIT_FIRST_LOAD	Load the first A/B K tile before computation can start.
S_START_COMPUTE	Pulse the wavefront feeder start signal.
S_RUN	Run the systolic array for the active K tile.
S_WAIT_BUFFER	Stall if the next buffer is not ready.
S_WRITE_REQ/S_WRITE_WAIT	Write the completed C tile back to memory.
S_NEXT_TILE	Advance (m_0, n_0) according to the selected dataflow traversal.
S_DONE	Signal completion.

This makes the hardware explicitly control what a CPU/GPU cache hierarchy normally hides: tile staging, reuse, scheduling, and writeback.

3.4.6 Ping-Pong Buffers and Double Buffering

Each tiled accelerator contains two local tile buffers:

$$\text{tile_a}[2][R][K_T], \quad \text{tile_b}[2][K_T][C].$$

One buffer can feed the array while the other buffer is being filled with the next K tile. With double buffering disabled, the schedule is serial:

$$\text{load} \rightarrow \text{compute} \rightarrow \text{load} \rightarrow \text{compute}.$$

With double buffering enabled, the first K tile is still loaded serially, but later loads can overlap the current compute phase:

$$\text{compute current tile} \parallel \text{prefetch next tile}.$$

Thus, in steady state, the time per non-final K tile becomes approximately

$$\max(T_{\text{compute}}, T_{\text{load}})$$

rather than

$$T_{\text{compute}} + T_{\text{load}}.$$

This is a key part of making the design behave like an ASIC-style streaming accelerator: the goal is not merely to build a large array, but to keep that array continuously fed.

3.4.7 Dataflow Modes: OS, WS, and IS

The hardware and simulator support three dataflow modes:

- **Output-stationary (OS).** The C partial sum remains in the PE accumulator. Both A and B tiles are reloaded for each output tile. This minimizes partial-sum traffic and is the most direct GEMM mapping.
- **Weight-stationary (WS).** B is treated as the stationary operand. The controller traverses n tiles outside m tiles so that, for a fixed n tile and K tile, the B tile can remain in the ping-pong buffers while multiple m tiles stream new A data. This reduces repeated B reads.
- **Input-stationary (IS).** A is preserved across multiple n tiles for the same m tile, reducing repeated A reads. This provides a useful contrast to WS because it changes which operand benefits from reuse.

These modes do not change the arithmetic count; they change memory traffic and scheduling.

3.4.8 Memory Hierarchy Model

The memory interface is modeled as a ready/valid request-response port with parameterized read latency, write latency, issue gaps, maximum outstanding requests, banks, row size, row-hit latency, row-miss penalty, and optional row-buffer behavior. The Python model includes cache-like profiles and a DRAM-like profile, which is identical to the one used for RTL simulation.

Profile	Read Lat.	Write Lat.	Max Out.	Banks	Row Words	Row Buffer
L1-like	4	1	64	16	64	off
L2-like	12	4	32	8	128	off
LLC-like	36	12	16	4	512	off
DRAM-like	120	60	8	4	1024	on

The cache-like profiles treat read/write latency as hit latency. The DRAM-like model additionally estimates row hits and row misses based on contiguous access runs. For A tiles, each row contributes a contiguous run of roughly K_T words; for B tiles, each row contributes a contiguous run of roughly C words. This lets the simulator approximate when the accelerator is memory-bound versus compute-bound.

3.5 Cycle-Approximate Performance Estimator

Why an analytical simulator was needed. Full VCS simulation is appropriate for validating correctness and debugging off-by-one timing bugs, but it becomes too slow for broad sweeps over large arrays and 1024×1024 GEMMs. Therefore, we built `estimate_perf.py`, a cycle-approximate performance model that mirrors the RTL controller, feeder, PE array timing, dataflow modes, and memory latency model. This shifted the final hardware component from an FPGA-only implementation to an **ASIC-style microarchitectural exploration**: the RTL defines the design, and the simulator evaluates design points that would be expensive to simulate cycle-by-cycle.

What the estimator models exactly vs approximately. The compute wavefront is modeled directly. For a K tile of active depth K_a on an $R \times C$ array, the run length is

$$T_{\text{run}}(K_a) = K_a + R + C - 2.$$

The controller also accounts for state-boundary cycles such as tile begin and start-compute. The memory model is approximate: it estimates load/store time using request count, latency, issue gap, maximum outstanding requests, row-buffer locality, and a calibration parameter called `memory_efficiency`. This gives a lower-bound style estimate when memory is idealized and can be made more conservative by reducing the efficiency parameter.

Metrics produced. The simulator records cycles, compute cycles, memory stall cycles, prefetch cycles, compute-wait cycles, read/write counts, bytes moved, loaded K tiles, output tiles, arithmetic intensity, PE utilization, effective GOP/s, compute GOP/s, and peak GOP/s. We report the accelerator numbers as GOP/s or FLOP-equivalent GOP/s because the RTL datapath uses integer MACs, while CPU/GPU baselines use floating-point arithmetic. For comparison to GFLOP/s baselines, one MAC is still counted as two operations.

For a throughput-oriented accelerator, the dominant question is whether each PE can accept one operation per cycle and whether the memory system can keep the array fed. A fully pipelined floating-point FMA would have a longer latency than an integer MAC, but it can still sustain an initiation interval of one operation per cycle. Therefore, for large GEMM workloads where steady-state throughput dominates fill/drain latency, an integer MAC is a reasonable proxy for studying array-level throughput. To avoid overstating the result, we report the accelerator’s arithmetic rate as GOP/s or “FP-equivalent GOP/s” rather than claiming a bit-exact measured GFLOP/s implementation.

4 RESULTS

4.1 Experimental Setup

We evaluated on the following machines:

- **CPU:** Intel Core i7-9700 (8 cores / 8 threads), base clock 3.0 GHz (max turbo 4.7 GHz), 16 GiB RAM.
- **GPU:** NVIDIA GeForce RTX 2080 (8 GiB), driver 550.67; CUDA 12.4; GPU code compiled with CUDA toolkit 11.7.
- **Accelerator model:** SystemVerilog RTL used for functional validation and a Python cycle-approximate simulator used for architecture sweeps. The simulated accelerator frequency is parameterized. Unless otherwise stated, architectural sweeps use a tiled systolic-array model with double buffering, a weight-stationary dataflow, and an L1-class on-chip memory model. Large-array sweeps should be interpreted as ASIC-style architectural projections rather than FPGA implementation results.

4.2 Metrics

We report throughput in GFLOP/s-equivalent using the standard GEMM convention of $2MNK$ operations. Since the RTL PE uses integer MACs for simplicity, the accelerator numbers should be interpreted as *projected fully-pipelined MAC throughput* rather than bit-exact IEEE-754 floating-point silicon performance. This is still the relevant architectural metric for our study: the goal is to evaluate whether the systolic dataflow can sustain high MAC throughput, keep PEs utilized, and hide memory latency through tiling and buffering. A fully pipelined floating-point FMA would add pipeline depth, but if its initiation interval is one, it would still preserve one MAC per PE per cycle in steady state.

For the accelerator model, we distinguish:

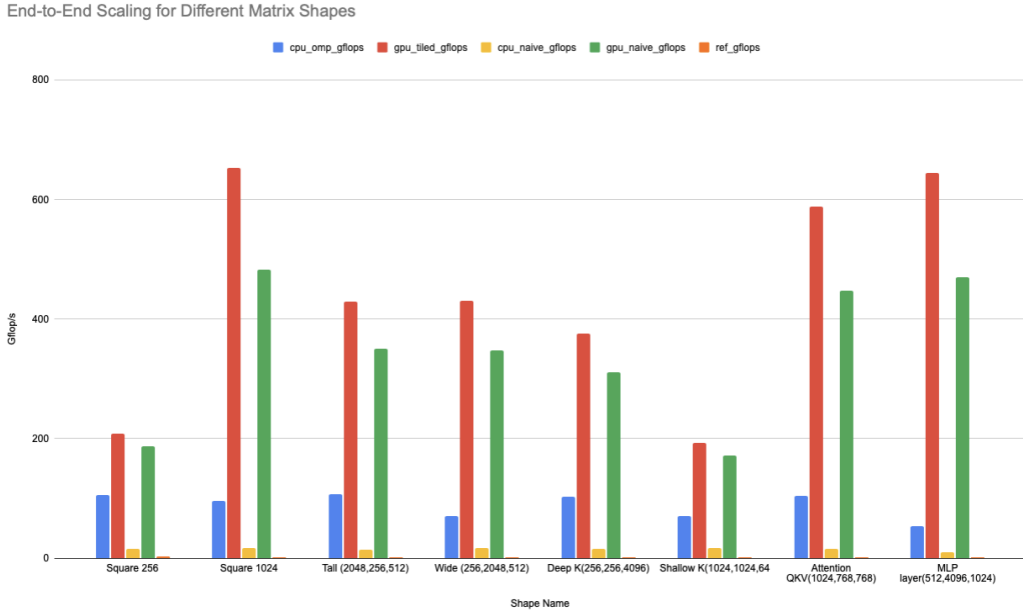
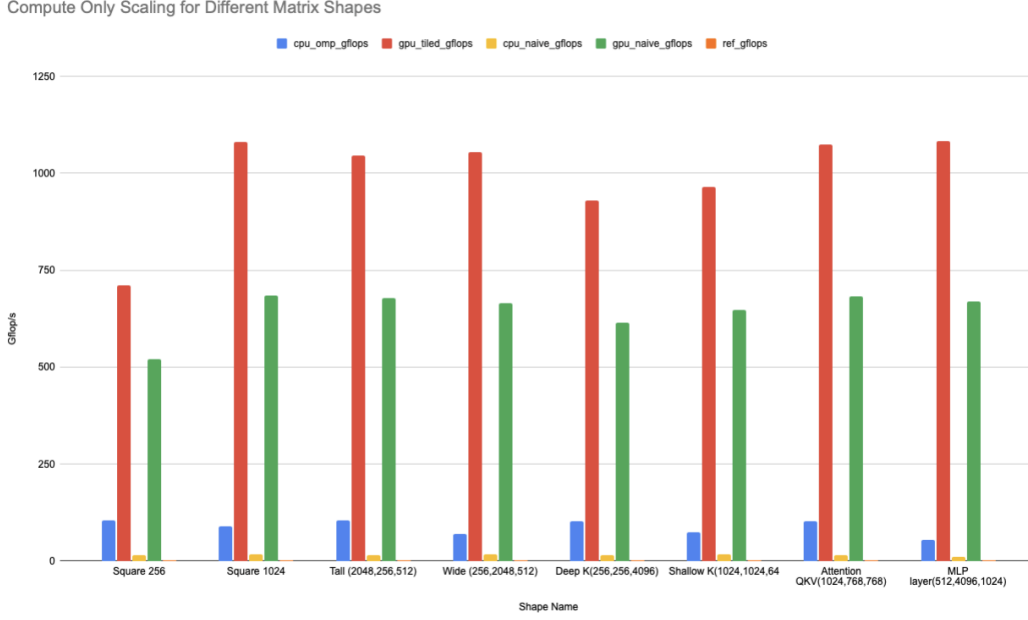
- **Effective throughput:** total GEMM operations divided by the full modeled execution time, including memory stalls, tile loading, compute, prefetch overlap, and writeback.
- **Compute throughput:** arithmetic throughput during the compute phase, isolating the quality of the systolic compute fabric from the backing memory model.
- **Peak throughput:** the ideal compute roof,

$$\text{Peak GFLOP/s} = \frac{2 \cdot R \cdot C \cdot f_{\text{MHz}}}{1000},$$

where $R \times C$ is the number of PEs and each PE can issue one MAC per cycle.

This distinction is important because the accelerator microarchitecture is highly throughput capable, but the achieved end-to-end rate depends heavily on whether the memory system can feed the array.

4.3 CPU and GPU Baseline Results



The CPU and GPU baselines establish reference points for conventional architectures across several representative GEMM shapes (balanced squares, tall/wide batches, shallow- K , and transformer-style projections).

On the balanced square case ($1024 \times 1024 \times 1024$), `cpu_omp` reaches about 89.6 GFLOP/s in compute-only mode versus about 16.2 GFLOP/s for `cpu_naive`, i.e., roughly $5.5\times$ faster. However, CPU throughput is

not uniform across shapes: on the wide batch/feature case ($256 \times 2048 \times 512$), `cpu_omp` drops to about 70.6 GFLOP/s while `cpu_naive` stays near 16.4 GFLOP/s, highlighting sensitivity to access patterns and blocking effectiveness beyond raw parallelism.

The GPU baselines are substantially stronger in compute-only mode, but the compute-only versus end-to-end gap varies strongly with problem shape because transfers amortize differently. For example, on $1024 \times 1024 \times 1024$, `gpu_tiled` is about 1081 GFLOP/s compute-only versus about 653 GFLOP/s end-to-end ($\approx 1.66\times$), whereas on shallow- K ($1024 \times 1024 \times 64$) it is about 964 GFLOP/s compute-only versus about 192 GFLOP/s end-to-end ($\approx 5.0\times$). Similarly, on an attention-style QKV projection ($1024 \times 768 \times 768$), `gpu_tiled` is about 1074 GFLOP/s compute-only versus about 587 GFLOP/s end-to-end ($\approx 1.83\times$). These gaps indicate that host/device movement and launch/synchronization overhead can dominate for small- K or otherwise “thin” GEMMs, even when the kernel itself is fast.

4.4 Accelerator Throughput versus Square Matrix Size

Figure 5 shows the modeled effective throughput of the tiled systolic accelerator for square GEMM. Throughput increases with matrix size because larger matrices amortize the fixed costs of tile setup, wavefront fill/drain, and writeback. The trend is clearest for the larger arrays: the 64×64 array improves from roughly 12.5 GFLOP/s at $N = 128$ to about 23–24 GFLOP/s at $N = 1536$ under the L1-class memory model. Smaller arrays saturate earlier because their compute capability is lower, so memory and orchestration overheads are less hidden by useful compute.

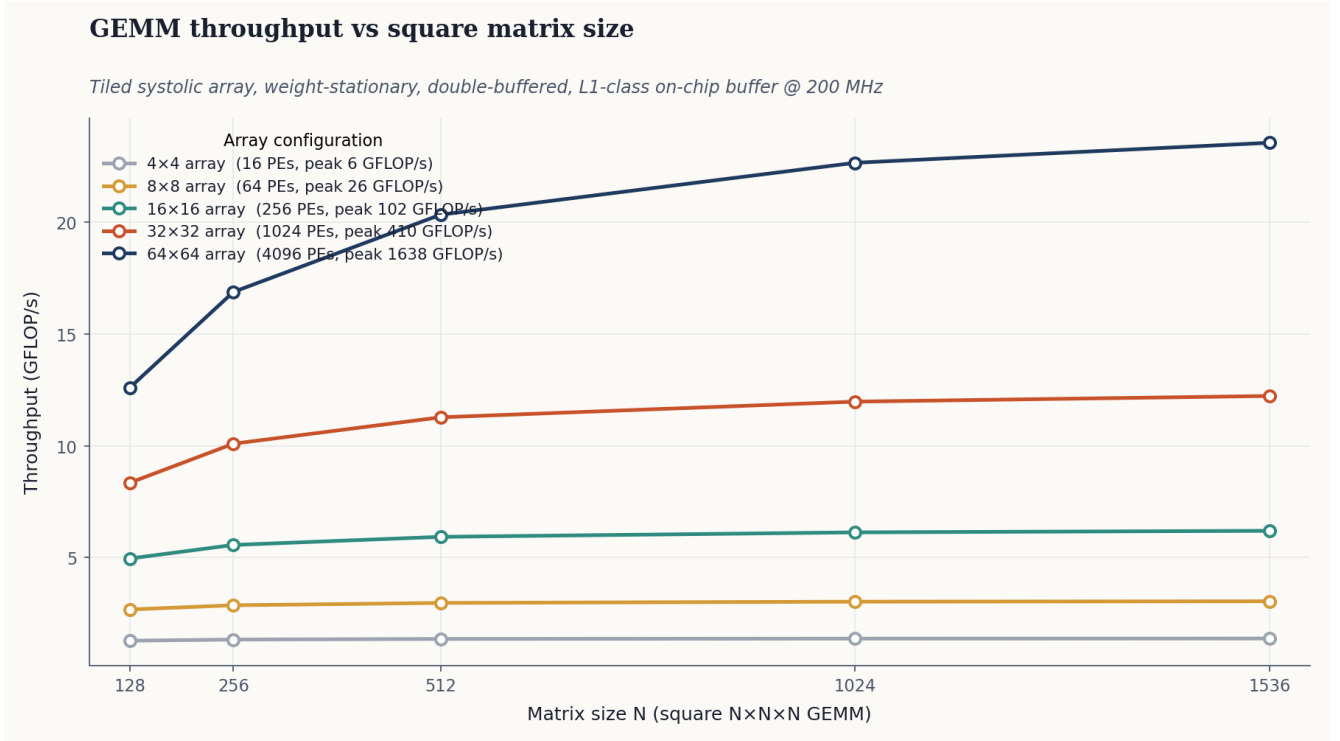


Figure 5: Effective accelerator throughput versus square GEMM size. Larger matrices better amortize tile setup and wavefront fill/drain overhead.

The important interpretation is that the architecture scales correctly: increasing PE count increases the available compute roof, and increasing problem size improves utilization. However, the absolute effective throughput remains far below the theoretical peak for large arrays. For example, at 200 MHz, a 64×64 array has a peak compute roof of approximately 1638 GFLOP/s-equivalent, but the achieved effective throughput in this memory-limited configuration is only tens of GFLOP/s. This gap is not because the systolic array is ineffective; rather, it shows that the array is starved by the modeled memory system. The compute fabric can consume far more data per cycle than the L1-class backing model supplies.

4.5 Throughput Across Realistic Workload Shapes

Figure 6 evaluates representative dense, batched, and attention-style GEMM shapes. The 32×32 array consistently outperforms the 16×16 and 8×8 arrays because it has more spatial parallelism. Most shapes reach roughly 10–12 GFLOP/s effective throughput for the 32×32 array, while the shallow- K case performs worse because the reduction dimension is too short to amortize wavefront overhead and tile setup.

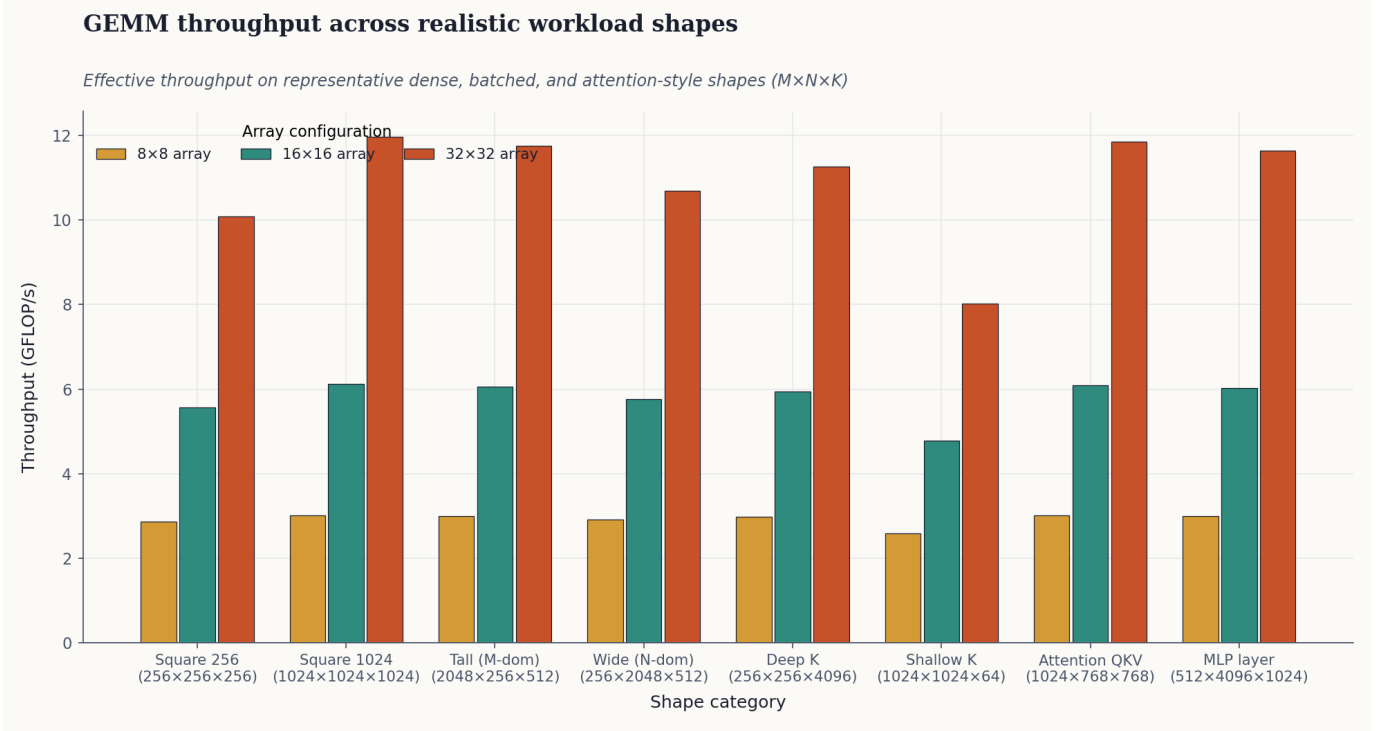


Figure 6: Effective throughput across representative GEMM shapes. The accelerator performs best when the workload has enough K depth and enough output tiles to keep the array busy.

This result supports the design choice of a tiled systolic array. Workloads such as attention QKV projections and MLP layers are naturally GEMM-heavy and expose enough structured reuse to benefit from the architecture. The main weak case is small or shallow- K GEMM, where there is not enough work per tile to fully amortize the pipeline fill/drain and memory staging overhead.

4.6 Dataflow Strategy: Output- vs. Weight- vs. Input-Stationary

Figure 7 compares output-stationary, weight-stationary, and input-stationary dataflows for the same 32×32 array. Output-stationary performs substantially worse in this model, reaching roughly 6 GFLOP/s, while weight-stationary and input-stationary reach around 11–12 GFLOP/s for large square matrices.

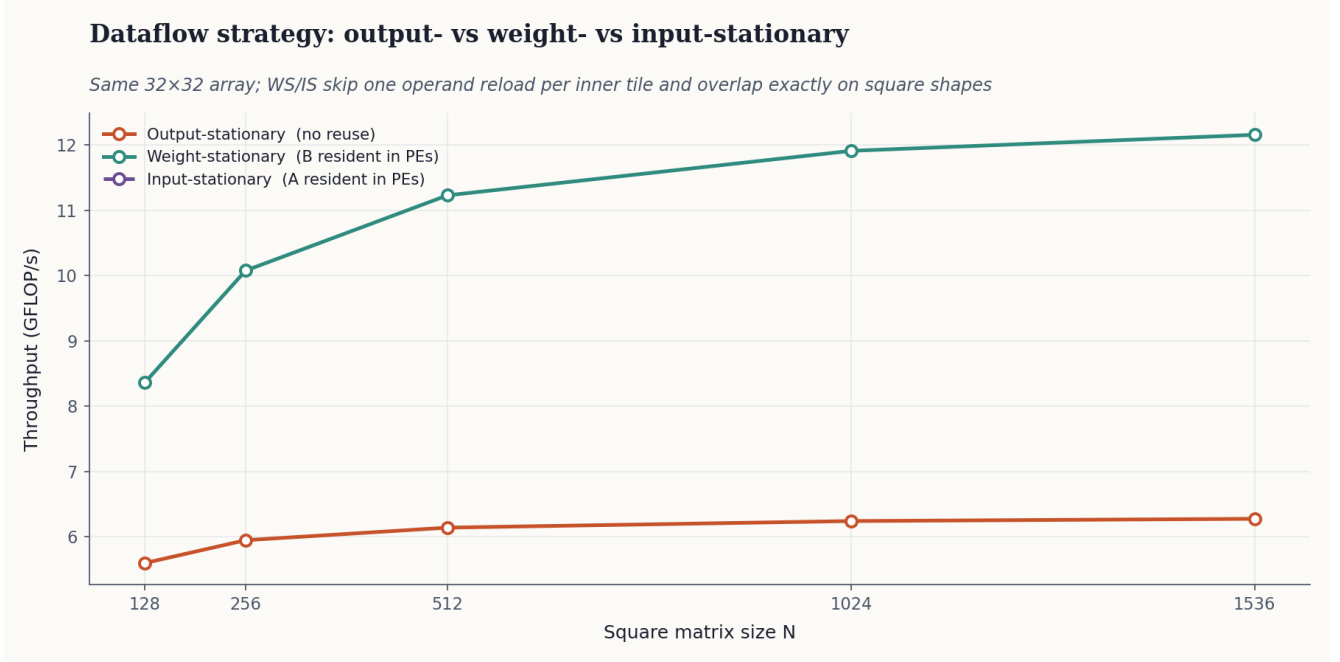


Figure 7: Dataflow comparison for a fixed 32×32 array. Weight-stationary and input-stationary reuse one operand across inner tiles and therefore reduce memory traffic relative to output-stationary.

The reason is that output-stationary keeps the partial sums local, but it still reloads both A and B tiles for each output tile. In contrast, weight-stationary and input-stationary reduce operand reloads by keeping one input resident across neighboring tiles. On square GEMM, the two reuse modes overlap almost exactly because the two dimensions are symmetric. This result is one of the strongest architectural takeaways: the PE array itself is not the bottleneck; data reuse is. Choosing a dataflow that reduces memory traffic nearly doubles effective throughput in the tested configuration.

4.7 Double Buffering and Prefetch Overlap

Figure 8 shows the impact of double buffering. Across the tested array sizes and matrix sizes, enabling double buffering improves throughput by approximately 4–22%. The benefit is largest for smaller arrays and smaller matrices because memory load time and tile transition overhead account for a larger fraction of total execution time. For larger arrays, the relative improvement is smaller because the memory system becomes more fundamentally bandwidth-limited rather than merely latency-limited.

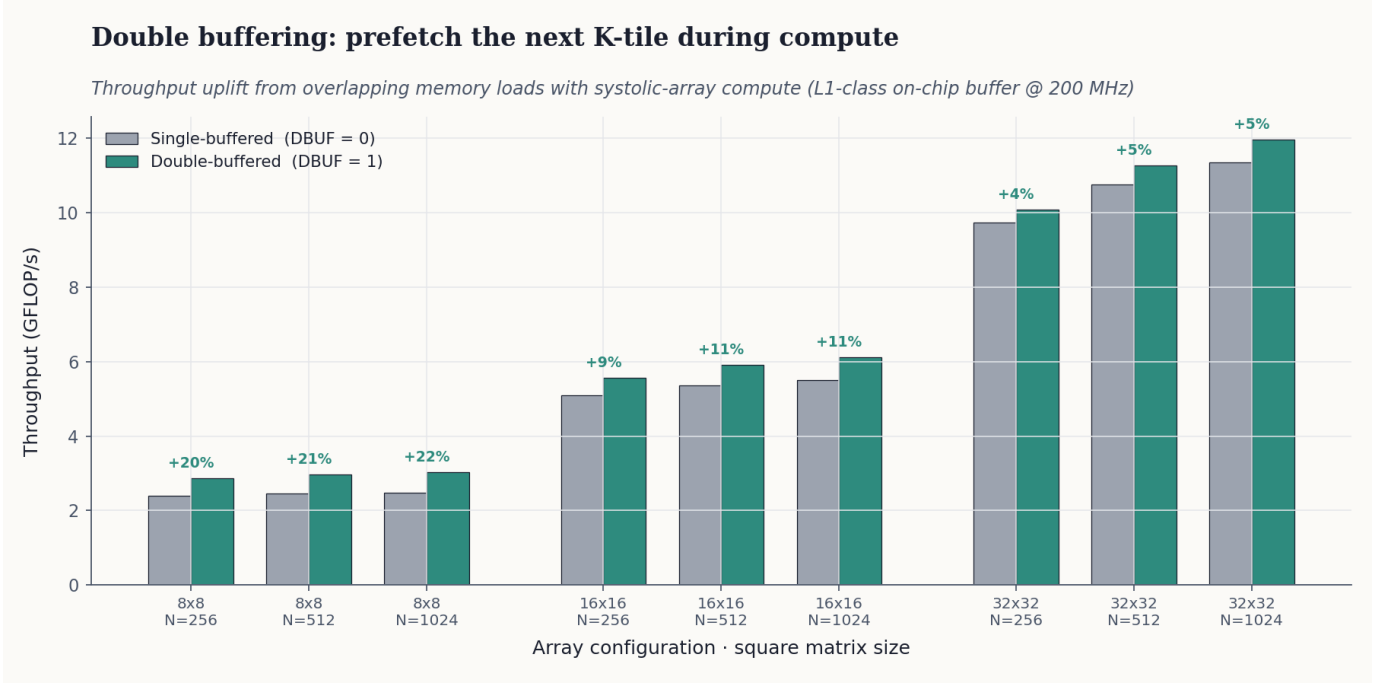


Figure 8: Double buffering improves throughput by prefetching the next K tile while the current tile is being computed.

This validates the ping-pong buffering strategy. Without double buffering, the accelerator follows a more serialized pattern:

load tile $\rightarrow$ compute tile $\rightarrow$ load next tile.

With double buffering, the steady-state cost becomes closer to:

$\max(\text{load next tile}, \text{compute current tile}),$

which hides some memory latency behind useful systolic-array work. The fact that double buffering helps but does not close the gap to peak throughput indicates that the remaining bottleneck is not just load latency; it is sustained memory bandwidth.

4.8 Memory Hierarchy Sensitivity

Figure 9 directly isolates the memory bottleneck by holding the array fixed and varying the backing memory model. The same 128×128 double-buffered, weight-stationary accelerator reaches about 6 GFLOP/s with an L1/L2-class on-chip memory model, but drops to roughly 2.7 GFLOP/s with an LLC-class model and about 2.0 GFLOP/s with a DRAM-like model.

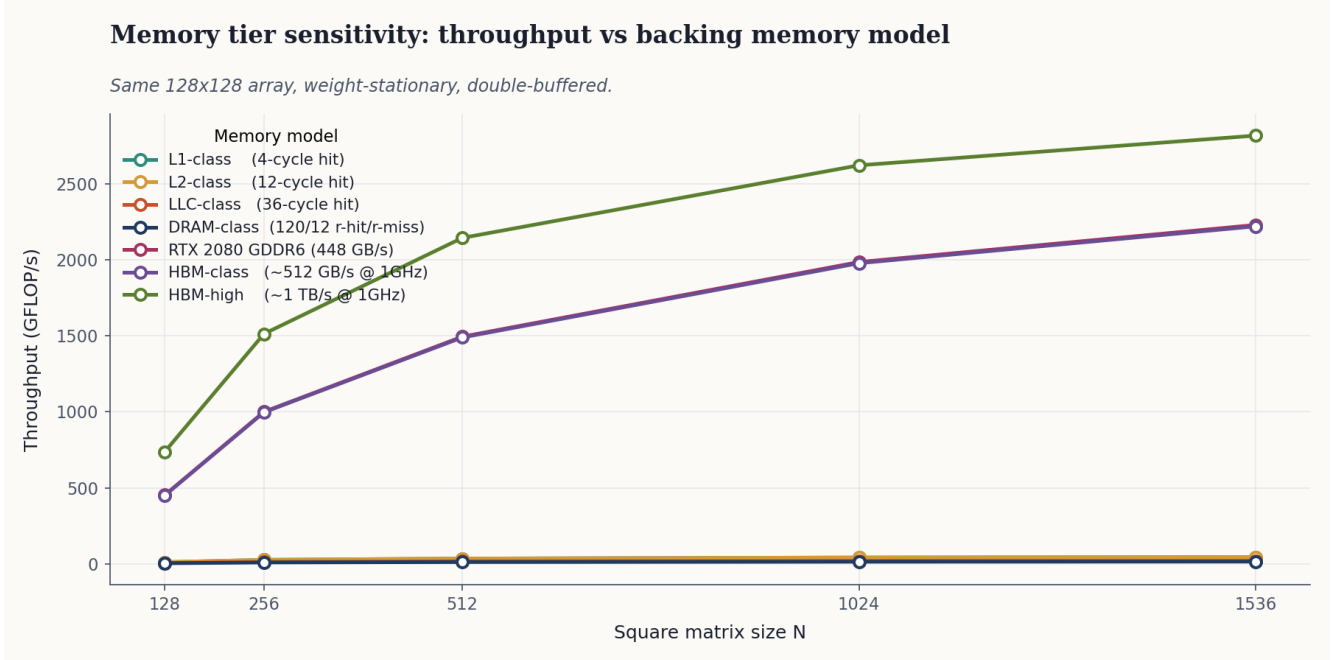


Figure 9: Memory hierarchy sensitivity for a fixed 128×128 array (similar size to Google’s newer TPUs). Throughput is highly sensitive to memory latency and bandwidth, confirming that the systolic array is memory-limited in end-to-end execution.

This figure is the clearest evidence that the design is fundamentally memory-limited. The compute fabric does not change across the curves; only the memory model changes. Therefore, the large throughput drop under LLC/DRAM-like memory is caused by the inability to supply operands quickly enough. This is exactly why modern systolic-array accelerators are paired with extremely high-bandwidth memory systems, large on-chip SRAMs, many banks, and carefully scheduled data reuse. In a real modern ASIC, using HBM or a high-bank-count scratchpad would directly attack the bottleneck shown in this figure.

4.9 Roofline Analysis

Figure 10 summarizes the same conclusion in roofline form. The dotted horizontal lines show the compute roofs of different array sizes, while the dashed diagonal line shows the active memory bandwidth roof. Most achieved points lie well below their array compute roofs and near the memory-limited region, especially for larger arrays. This means that simply adding more PEs increases the theoretical peak, but achieved throughput only improves if the memory system scales with the compute fabric.

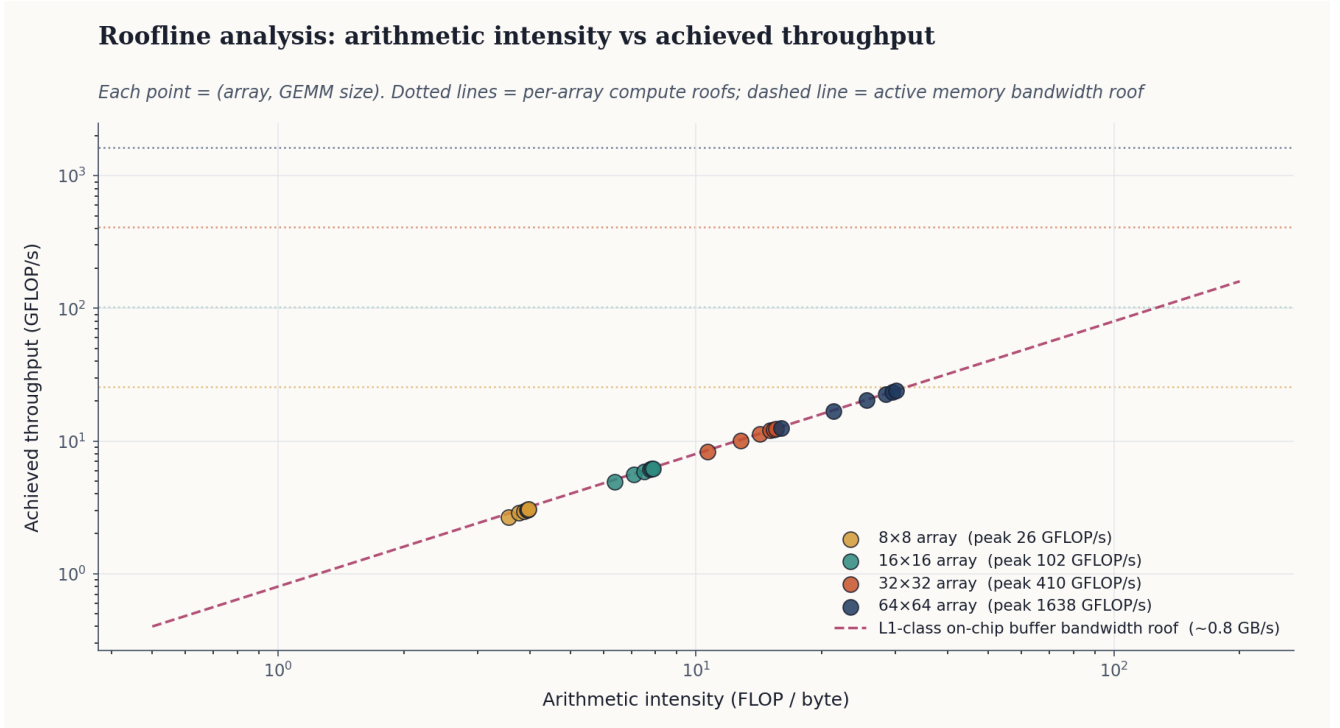


Figure 10: Roofline analysis. Most achieved points are far below their compute roofs, showing that the modeled accelerator is limited primarily by memory bandwidth rather than MAC capacity.

This roofline plot is also the strongest argument for the potential of the design. The 64×64 array has a compute roof above 1.6 TFLOP/s-equivalent at 200 MHz, and larger arrays have even higher roofs. The achieved points do not reach these roofs because the memory system is too weak, not because the systolic array lacks compute parallelism. If the memory bandwidth roof were shifted upward through HBM, more aggressive SRAM banking, or wider memory ports, the achieved points would move closer to the array compute roofs. In that regime, the architecture can exceed the measured RTX 2080 compute-only throughput.

4.10 PE Utilization and Pipeline Fill/Drain Effects

Figure 11 shows PE utilization during the compute phase as a function of matrix size. Small matrices underutilize large arrays because the wavefront has not yet reached steady state before the computation is nearly over. For example, a 64×64 array has almost no useful utilization at $N = 16$ and only begins to approach the utilization of smaller arrays once N becomes large enough to amortize fill/drain overhead.

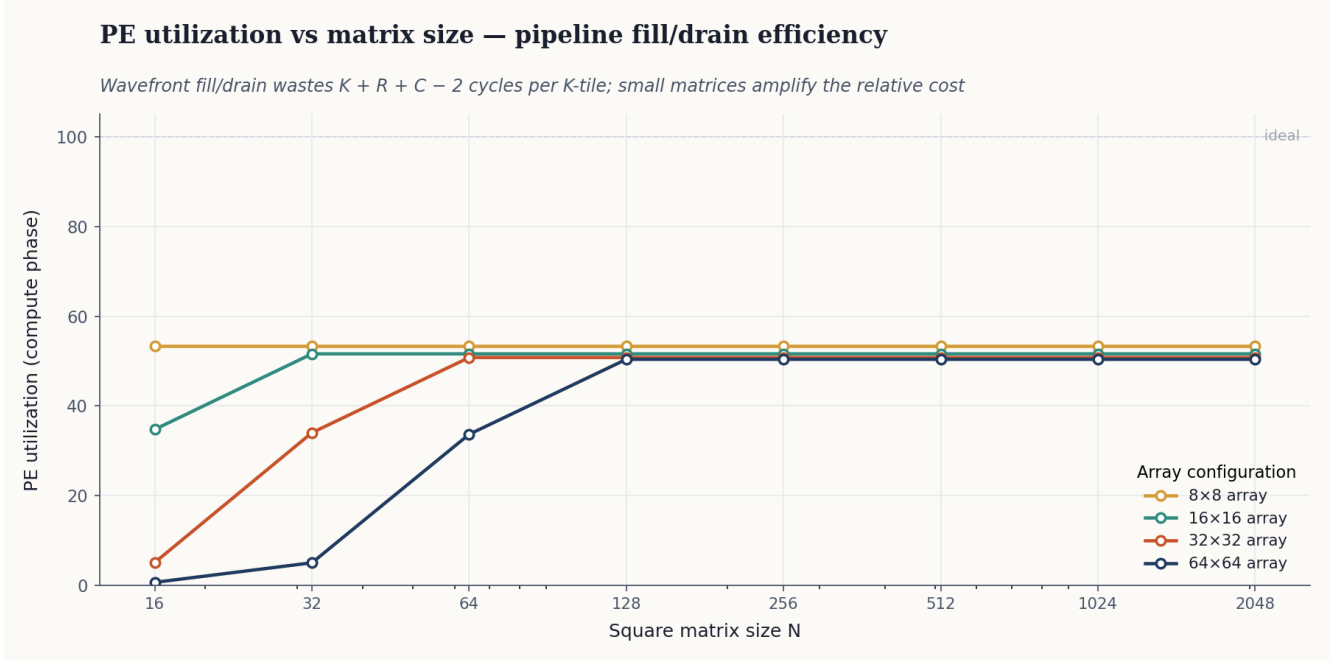


Figure 11: PE utilization versus matrix size. Large arrays require large enough matrices to amortize wavefront fill/drain overhead and keep PEs busy.

The utilization curves plateau around roughly 50%, rather than 100%, because this metric includes the wavefront schedule and tile-level effects rather than only ideal steady-state cycles. This is still a useful result: it shows that large arrays are only beneficial when the workload is large enough. For tiny GEMMs, large arrays waste hardware; for large GEMMs, they provide substantial throughput headroom.

4.11 Cycle Decomposition

Figure 12 decomposes modeled runtime into compute and memory stall fractions. The dominant component is memory stall, especially as the array size increases. For larger arrays, the compute fraction becomes very small because the array can complete each tile quickly, but the memory system cannot supply the next tile at the same rate.

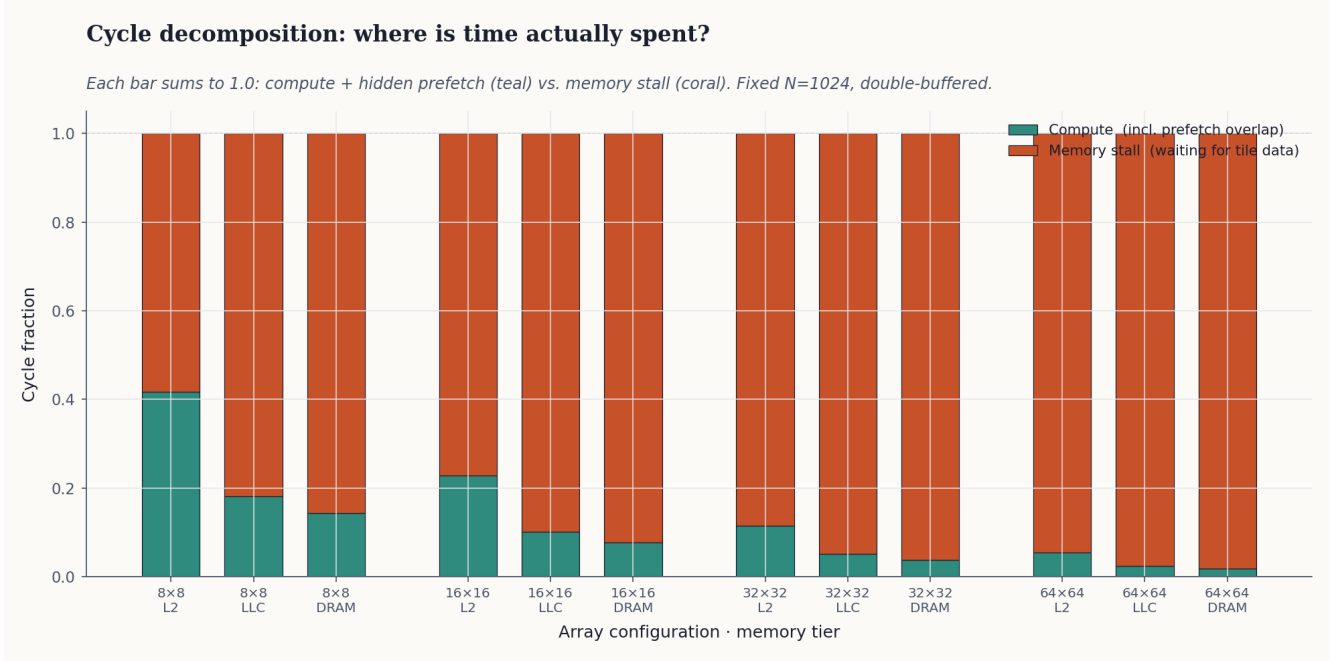


Figure 12: Cycle decomposition for fixed $N = 1024$. Memory stall dominates total runtime, especially for larger arrays and slower memory tiers.

This is the most direct explanation for why the end-to-end throughput appears low relative to peak. The accelerator is not spending most of its time doing useful MACs; it is waiting for data. Double buffering hides some of the load latency, but once the array becomes large enough, the required operand bandwidth grows faster than the modeled memory system can support. This further supports the HBM argument: the correct way to unlock this architecture is not to redesign the PE, but to pair the PE array with a memory system capable of feeding it.

4.12 Scaling with PE Count

Figure 13 compares ideal peak throughput against achieved throughput as PE count grows. The ideal curve scales linearly with PE count because each additional PE contributes one MAC per cycle. However, achieved throughput grows much more slowly and eventually becomes a small fraction of peak: roughly 21% of peak for the smallest tested array, falling to about 1% for the 64×64 array.

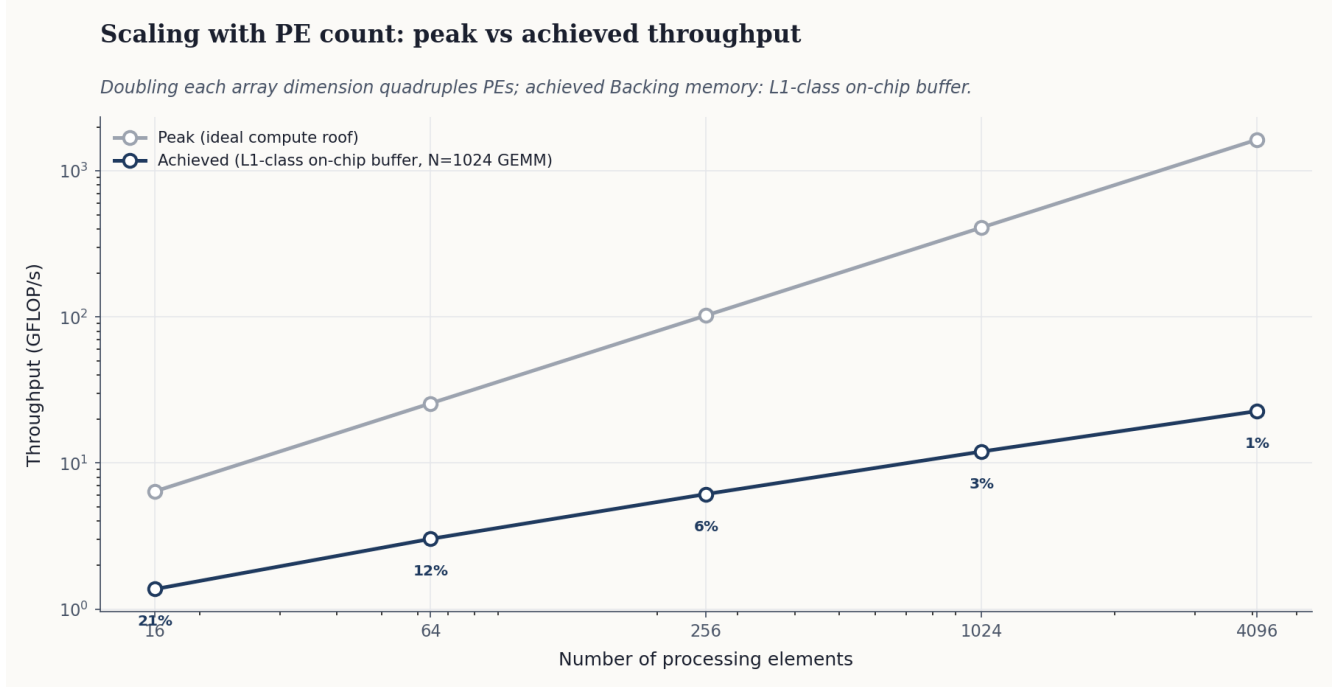


Figure 13: Peak versus achieved throughput as PE count scales. Peak compute grows linearly with PE count, but achieved throughput is limited by memory bandwidth.

This figure should not be read as a failure of scaling. Instead, it shows that compute scaling has outpaced memory scaling. The systolic design creates enormous compute demand: a 64×64 array can theoretically perform 4096 MACs per cycle. Feeding that array requires a memory system with enough bandwidth to deliver operands at the required rate. The modeled L1-class memory is not sufficient to sustain this at high utilization. Modern accelerators solve exactly this problem using HBM stacks, wide NoCs, heavily banked SRAMs, and explicit dataflow scheduling. Under those assumptions, the same compute fabric can operate much closer to peak.

4.13 Compute-Only Accelerator Throughput

To separate memory effects from the quality of the compute fabric, Figures 14 and 15 show compute-only throughput. These figures are the best measure of the systolic array itself. The compute-only results scale much more aggressively with array size. A 64×64 array reaches hundreds of GFLOP/s-equivalent, a 128×128 array reaches multiple TFLOP/s-equivalent, and a 256×256 array reaches over 10 TFLOP/s-equivalent on large enough problems. As a direct cross-platform comparison: at compute-only throughput with a 128×128 at 200 MHz, the accelerator projects to approximately 6,5546 GFLOP/s-equivalent, representing roughly 6x the measured `gpu_tiled` compute-only throughput of 1,072 GFLOP/s and 89x the `cpu_omp` throughput of 73.6 GFLOP/s. These ratios are for the compute phase only; end-to-end the GPU remains ahead due to the memory bandwidth.

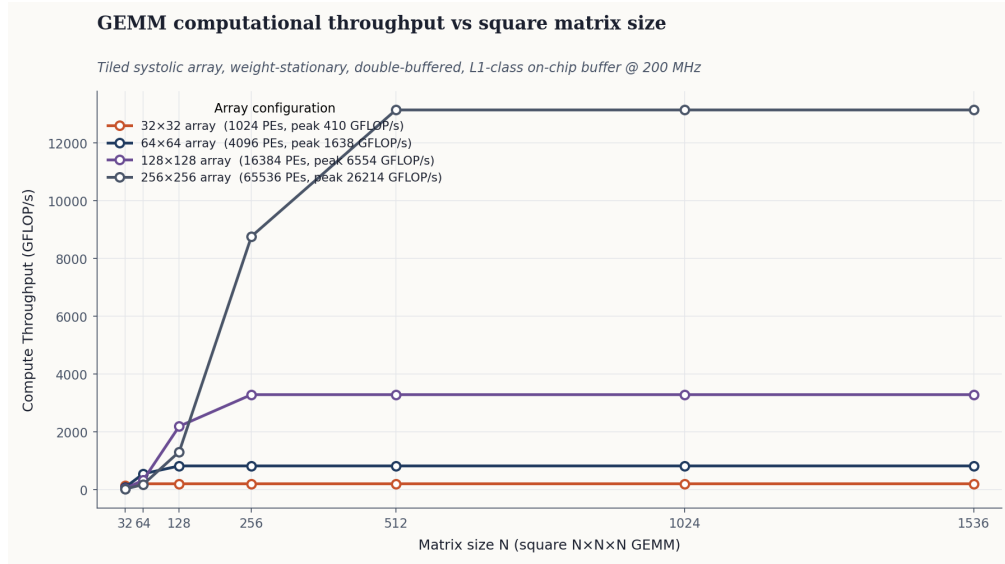


Figure 14: Compute-only accelerator throughput versus square matrix size. Once memory stalls are removed, large systolic arrays scale to GPU-class and beyond-GPU-class arithmetic rates.

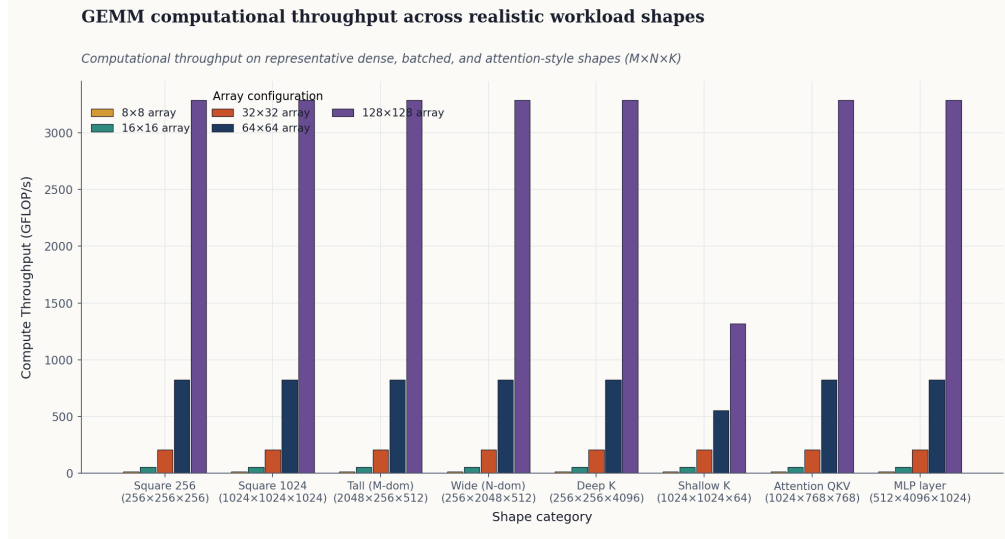


Figure 15: Compute-only accelerator throughput across workload shapes. Large arrays provide very high arithmetic throughput when the workload is large enough to utilize the array.

These results show the main positive conclusion of the project: the systolic-array microarchitecture is highly effective as a throughput engine. The compute fabric itself can exceed the measured GPU compute-only baseline when scaled to ASIC-style array sizes. The reason the effective end-to-end throughput is lower is not the PE datapath or the wavefront schedule; it is the memory hierarchy. Therefore, with a modern memory system such as HBM, which provides far higher bandwidth than the simple modeled L1/LLC/DRAM tiers, this architecture has a clear path to surpassing GPU-class throughput on large GEMM workloads.

4.14 What Limited Speedup

The limiting factor differs by platform.

Transfer overhead on GPU. At $N = 1024$, `gpu_tiled` improves from about 655.0 GFLOP/s in end-to-end mode to about 1081.6 GFLOP/s in compute-only mode, a ratio of about $1.65\times$. For `gpu_naive`, the same ratio is about $1.43\times$ (478.2 to 684.5 GFLOP/s). This shows that GPU performance is affected not only by kernel arithmetic throughput, but also by host-device transfer overhead and launch/synchronization costs.

Cache and scheduling effects on CPU. At $N = 1024$ in compute-only mode, `cpu_omp` with tile size 64 reaches about 73.6 GFLOP/s, while `cpu_naive` reaches about 16.1 GFLOP/s. This $4.57\times$ improvement confirms the importance of blocking and parallelism. However, the fact that tile 128 underperforms tile 64 shows that CPU performance is sensitive to cache behavior, scheduling overhead, and memory hierarchy effects.

Memory bandwidth on the systolic accelerator. For the systolic accelerator, the compute fabric scales extremely well, but achieved effective throughput is limited by memory bandwidth. This is shown consistently across the memory sensitivity plot, roofline plot, cycle decomposition, and peak-versus-achieved scaling plot. Larger arrays have enormous theoretical compute roofs, but they also demand much higher operand bandwidth. In our model, the backing memory system cannot feed the larger arrays fast enough, so the arrays spend most cycles stalled or waiting for tile data.

This is also why the design is promising: the bottleneck is not a lack of parallelism or a poor compute mapping. The bottleneck is the memory system. Modern systolic-array accelerators are typically paired with HBM or very wide, heavily banked on-chip SRAM systems precisely to solve this problem. With an HBM-class memory subsystem and enough banking to sustain the operand streams, the architecture can move significantly closer to its compute roof and surpass the GPU baseline on large GEMM workloads.

Was the accelerator target the right choice? For large, regular GEMM with sufficient matrix size to amortize fill/drain overhead, the systolic-array architecture is well-matched to the workload: the computation is entirely data-parallel across output elements, the access pattern is highly structured, and the reuse pattern maps directly onto the wavefront schedule. The GPU, by contrast, provides higher raw memory bandwidth and a mature software stack, which is why it outperforms the accelerator on end-to-end throughput in our study. However, the GPU’s advantage is a memory-system advantage, not a compute-architecture advantage — the compute-only results in 4.13 show that the systolic array can match or exceed GPU arithmetic throughput at ASIC-scale array sizes. For a deployment scenario with HBM-class memory and a clock frequency above 500 MHz (achievable in modern ASIC processes), the accelerator target is sound. At 200 MHz with an L1-class memory model, the CPU is the better practical choice for small matrices, the GPU is better end-to-end for all sizes tested, and the accelerator is competitive only in the compute-only regime. This makes the memory system, not the array architecture itself, the determining factor in platform choice.

4.15 Summary of Results

1. **The systolic-array compute fabric is strong.** Compute-only sweeps show that large arrays scale to GPU-class and beyond-GPU-class arithmetic throughput.
2. **Dataflow and buffering matter.** Weight-stationary/input-stationary reuse and double buffering substantially improve effective throughput by reducing reloads and hiding latency.
3. **Memory bandwidth is the dominant bottleneck.** End-to-end effective throughput is far below peak because the modeled memory system cannot feed large arrays fast enough. This is a system-level bottleneck, not a flaw in the systolic PE architecture.

Thus, the project demonstrates a successful tiled systolic-array accelerator microarchitecture. The remaining gap to peak performance is primarily a memory-system problem. A realistic modern ASIC implementation would address this with HBM, wider memory interfaces, more SRAM banking, and more aggressive operand reuse, enabling the compute fabric to approach its projected throughput and potentially exceed the measured GPU baseline for large GEMM workloads.

5 REFERENCES

- [1] NVIDIA Corporation, *CUDA C++ Programming Guide*, v12.8, 2025.
- [2] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Version 5.2*, Nov. 2021.
- [3] H. T. Kung, “Why Systolic Architectures?,” *Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982.

6 WORK DISTRIBUTION AND CREDIT

We split work equally (50-50). The breakdowns are as follows.

Albert:

- RTL Design for Output-Stationary Systolic Array Design
- RTL Design for Tiled Accelerators and GEMM Engine
- RTL Testbenches
- Synthesis and Timing Closure
- Hardware Level Sweeps

Raymond:

- RTL for Dataflow Mode Parameterization
- CPU and GPU Baselines
- Benchmark Harness
- Benchmarking Automation
- Plots and Data Analysis